Optimal Reciprocal Collision Avoidance for Robot Crowds: Implementation, Verification of Reciprocity and the Symmetric Deadlock

Mahim Trivedi
Python Robotics Projects, github.com/robofan45/orca-collision-avoidance
November 2026

***Abstract*—Optimal reciprocal collision avoidance (ORCA) lets many robots move through each other's way without communication or central planning. Each robot computes, for every neighbour, the half-plane of velocities that avoids a collision with it if both robots share the effort equally, and then picks the velocity closest to the one it would prefer among those allowed by all half-planes, by solving a small linear program. This report presents a compact implementation of ORCA, following the reference library RVO2, and evaluates it on the classic benchmark in which robots on a circle swap to the opposite point, so that all of them meet in the middle. Ten robots complete the swap in 11.8 s, only 10% slower than if each were alone, and never overlap; their closest approach is 1.000025 m against a contact distance of 1 m. A check on 16,535 random pairs of velocities confirms the reciprocity guarantee on which the method rests: when both robots choose from their half-planes, they never collide within the time horizon. The report then examines a failure that the guarantee does not cover. In a perfectly symmetric configuration, every robot's problem is symmetric about its own line to the goal, so the robots slow down and stall in a ring around the centre. Turning every preferred velocity a few degrees to the right breaks the symmetry, and the crowd flows like traffic in a roundabout; an ablation shows that one degree suffices. Finally, a crowd-size study shows the limit of the method: 16 robots finish in 14.2 s, while 24 jam without ever making the linear program infeasible.**

***Index Terms*—*multi-robot systems, collision avoidance, velocity obstacles, reciprocal collision avoidance, linear programming, decentralised control, crowd simulation*

I. Introduction

When many robots share a space, each must avoid the others while pursuing its own goal. Centralised planning for all robots at once scales poorly with their number and requires communication, so practical systems often let each robot decide for itself, reacting at every control step to the observed positions and velocities of its neighbours. The challenge of such decentralised methods is that the neighbours are not static obstacles: they react too.

Velocity obstacles capture the first half of the problem [1]. For a moving neighbour, the velocity obstacle is the set of velocities that would lead to a collision within a time horizon, and choosing a velocity outside it avoids the collision as long as the neighbour keeps its velocity. When both robots reason this way, however, both avoid the full collision, both overcorrect, and in the next step both see that their old velocities would have been safe and return to them, a reciprocal oscillation. Reciprocal velocity obstacles reduce the oscillation by letting each robot take half of the avoidance [2], and optimal reciprocal collision avoidance (ORCA) turns this idea into a convex problem with a guarantee [3]. Each robot computes, for each neighbour, a half-plane of velocities such that, if both robots choose from their half-planes, they cannot collide within the time horizon. The robot then chooses the velocity closest to its preferred one within all half-planes, which is a two-dimensional linear program that can be solved in expected linear time.

This report describes an implementation of ORCA and studies how it behaves in the circle-swap benchmark, in which all robots must pass through the centre at once. Its contributions are:

- a compact implementation of ORCA, including the half-plane construction, the incremental linear program and the fallback for crowds in which no velocity satisfies every half-plane, in about 300 lines of Python;

- a numerical verification of the reciprocity guarantee on 16,535 random pairs of velocities;
- an analysis of the deadlock that perfect symmetry causes, and an ablation of the small keep-right bias that resolves it;
- a crowd-size study that locates the limit of the method in this benchmark, where a jam forms although every linear program remains feasible.

II. Related Work

Fiorini and Shiller introduced velocity obstacles for motion planning among moving obstacles [1]. Van den Berg, Lin and Manocha observed the oscillations that arise when every agent treats the others as passive, and proposed reciprocal velocity obstacles, in which each agent takes half of the responsibility for avoiding a collision [2]. The hybrid reciprocal velocity obstacle further reduces the reciprocal dances in which two agents keep choosing the same side [4]. ORCA replaces the velocity obstacles by half-planes, which makes the set of allowed velocities convex and the choice of velocity a linear program, and proves that agents choosing from their half-planes are collision-free for the time horizon [3]; the authors' RVO2 library is the reference implementation followed here [5]. ORCA has been extended to robots with nonholonomic and other kinematic constraints [6], [7].

Other decentralised approaches include artificial potential fields [8] and the social force model of pedestrian dynamics [9], which add repulsive terms to the controller but give no collision guarantee; control barrier function filters, which provide guarantees for multi-robot systems through a quadratic program [10]; and learned policies trained with reinforcement learning [11]. The linear program at the heart of ORCA has two variables and is solved by the incremental algorithm of Seidel [12], described in the textbook of de Berg et al. [13], which adds one constraint at a time and runs in expected linear time when the constraints are taken in random order.

III. Problem Formulation

N robots, discs of radius $r = 0.5$ m, start evenly spaced on a circle of radius 8 m, and each must reach the point diametrically opposite, so that every straight path passes through the centre. Each robot controls its velocity directly, up to a maximum speed $v_{\max} = 1.5$ m/s, and observes the positions and current velocities of all other robots, but not their goals or intentions; there is no communication. Every control step of 0.1 s, all robots choose their new velocities from the same snapshot and then move. The task is solved when every robot is within 5 cm of its goal, and two robots collide if their centres come closer than $2r = 1$ m. Without the others, each robot would cover its 16 m in 10.7 s.

IV. Method

A. Velocity obstacles

For robots A and B at positions p_A and p_B, the velocity obstacle with time horizon τ is the set of relative velocities that cause a collision within τ:

$$VO^{\tau}_{(A|B)} = \{\, v \mid \exists t \in [0,\tau] : tv \in D(p_B - p_A, r_A + r_B) \,\} \tag{1}$$

where $D(c,\rho)$ is the open disc of radius ρ centred at c. Geometrically it is a cone with its apex at the origin, truncated by a disc of radius $\frac{r_A+r_B}{\tau}$ centred at $\frac{p_B-p_A}{\tau}$.

B. Reciprocal half-planes

If the current relative velocity $v_A - v_B$ lies inside the velocity obstacle, a collision is coming. Let u be the smallest change of the relative velocity that takes it to the boundary of the obstacle, and n the outward normal of the boundary at that point:

$$u = w^* - (v_A - v_B), \qquad w^* = \arg\min_{w \in \partial VO^{\tau}_{(A|B)}} \|w - (v_A - v_B)\| \tag{2}$$

ORCA lets each robot make half of this change. Robot A restricts its new velocity to the half-plane

$$\mathrm{ORCA}^{\tau}_{(A|B)} = \left\{ v \;\middle|\; \left(v - \left(v_A + \frac{1}{2}u \right) \right) \cdot n \geq 0 \right\} \tag{3}$$

and robot B to the mirror-image half-plane, with $-\frac{u}{2}$. If the relative velocity is already outside the obstacle, the same construction gives a half-plane that contains the current velocity and lets each robot use up at most half of the remaining margin to the obstacle. The central property of ORCA is reciprocity: if A chooses any velocity in its half-plane and B any velocity in its own, their relative velocity lies outside the velocity obstacle, so they do not collide within τ [3]. The boundary point w^* lies either on the truncating disc or on one of the two legs of the cone, and the implementation distinguishes the two cases as RVO2 does [5]. If two robots already overlap, which cannot happen in exact arithmetic but must be handled, the half-plane asks them to separate within one time step.

C. Choosing the velocity

Robot A chooses the velocity closest to its preferred velocity among those allowed by all neighbours and its speed limit:

$$v_A^{\mathrm{new}} = \arg\min_{v \in \mathrm{ORCA}^{\tau}_A} \left\| v - v_A^{\mathrm{pref}} \right\|, \; \mathrm{ORCA}^{\tau}_A = D(0, v_{\max}) \cap \bigcap_{B \neq A} \mathrm{ORCA}^{\tau}_{(A|B)} \tag{4}$$

The preferred velocity points at the goal at full speed. The problem has two variables, a convex objective and linear constraints apart from the speed limit, and is solved with the incremental algorithm [12], [13]. The constraints are added one at a time; as long as the current solution satisfies the next half-plane it remains the solution, and otherwise the new solution lies on that half-plane's boundary, where it is found by a one-dimensional problem over the segment of the boundary that satisfies all earlier constraints and the speed limit. As in RVO2, the half-planes are added in order of the distance to the neighbour, so that if the problem turns out to be infeasible, the constraints of the nearest neighbours have already been met.

D. Dense crowds

In a dense crowd the half-planes can have no common point within the speed limit. ORCA then chooses the velocity that violates the half-planes least, in the sense of the largest distance by which it lies outside any of them:

$$v^* = \arg\min_{\|v\| \leq v_{\max}} \max_{B \neq A} \delta_B(v) \tag{5}$$

where $\delta_B(v)$ is the distance by which v lies outside the half-plane of neighbour B, or zero. Geometrically, all boundaries are moved outwards at the same speed until they have a common point, which RVO2 computes with a three-dimensional linear program [5]; the implementation does the same by solving a sequence of two-dimensional programs on the lines along which two half-planes are violated equally.

E. Keeping right

Section VI-C shows that the circle swap, with its perfect symmetry, makes the robots stall. The remedy is a convention that drivers know well. Every preferred velocity is turned by a small angle θ_r to the right of the goal direction,

$$v_A^{\mathrm{pref}} = \frac{v_{\max} R(-\theta_r)(g_A - p_A)}{\|g_A - p_A\|} \tag{6}$$

where g_A is the goal and R a rotation, with $\theta_r = 5°$ in the demonstration. Within one step of its goal, a robot instead moves exactly onto it. Algorithm 1 summarises one control step of a robot.

Algorithm 1. One ORCA step of robot A

1: Compute the preferred velocity: towards the goal at full speed, turned θ_r to the right.
2: Sort the other robots by distance, and compute the ORCA half-plane for each from the current snapshot.
3: Solve the two-dimensional linear program for the velocity closest to the preferred one within all half-planes and the speed limit.
4: If it is infeasible, choose the velocity that minimises the largest violation of the half-planes instead.
5: After all robots have chosen, move every robot with its new velocity for one time step.

V. Implementation

The method is a single Python module of about 300 lines built on NumPy [14]. It follows the structure of RVO2 [5] closely, with the half-plane construction, the incremental linear program and the fallback as separate functions, so that each can be tested on its own. One step for ten robots, in which each robot builds nine half-planes and solves its linear program, takes 1.1 ms. The code grew out of the PythonRobotics collection [15] and follows its conventions: running the module animates the robots with their velocities and trails. Table I lists the parameters.

Six automated tests check the method. They assert that every robot reaches its goal; that no two robots ever overlap; that every speed stays within the limit; that two robots choosing velocities from their respective half-planes never collide within the time horizon, on random configurations, which tests the reciprocity guarantee directly; that the linear program agrees with a general-purpose optimiser on random problems; and that the crowd fallback splits the violation evenly between symmetric half-planes.

TABLE I
Parameters of the benchmark and the method

Parameter	Symbol	Value
Number of robots (demonstration)	N	10
Start circle radius; goals		8 m; the opposite points
Robot radius	r	0.5 m
Maximum speed	$v_{\max}$	1.5 m/s
Time horizon	τ	3 s
Control step	Δt	0.1 s
Keep-right angle	θ_r	5°
Goal tolerance; time limit		0.05 m; 40 s

VI. Experiments

A. Demonstration

Fig. 1 shows the paths of the ten robots. All of them reach their goals after 11.8 s, 1.1 s or 10% later than a lone robot would. Because every robot turns slightly to the right, the crowd does not meet head-on in the middle but circulates counter-clockwise around a free disc of about 1.6 m radius, like traffic in a roundabout, and leaves it towards the goals. Fig. 2 shows the closest distance between any two robots and the mean speed. The robots close in on each other for 6 s, reach their closest approach of 1.000025 m, 25 µm more than contact, and then separate; their mean speed dips to about 1.22 m/s while

they circulate. Table II summarises the run. The fallback for infeasible linear programs was never needed.

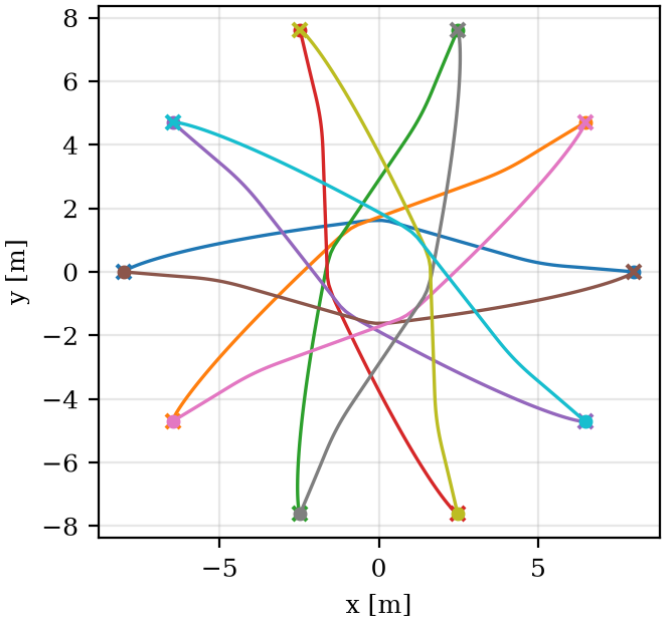


Fig. 1. Paths of the ten robots in the demonstration. Each robot starts at a dot and ends at the cross on the opposite side; the paths circulate counter-clockwise around the centre.

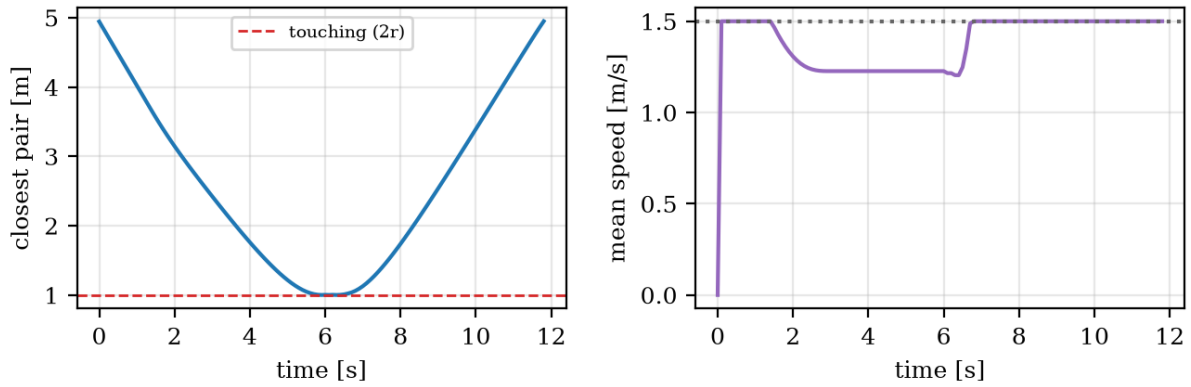


Fig. 2. Left: distance between the closest pair of robots; the dashed line marks contact. Right: mean speed of the robots; the dotted line marks the maximum speed.

TABLE II
SUMMARY OF THE DEMONSTRATION AND THE RECIPROCITY CHECK

Measure	Value
Time until every robot has arrived	11.8 s (a lone robot: 10.7 s)
Closest approach between two robots	1.000025 m (contact at 1 m)
Lowest mean speed of the crowd	1.22 m/s
Linear programs that needed the fallback	0
Time per step for ten robots	1.1 ms
Random velocity pairs checked for reciprocity	16,535
Closest approach within the horizon, any checked pair	1.005 m

The closest approach shows how ORCA works: it does not keep robots apart by a margin but lets them come as close as the half-planes allow, which in a crowd means sliding past each other at practically zero clearance. Any margin needed for sensing or control errors must be added to the robots' radii.

B. Reciprocity

The safety of ORCA rests on the reciprocity property of Section IV-B, which the implementation must reproduce exactly. To check it, 2000 random pairs of robots with random positions and velocities were generated. For each pair, both half-planes were computed, and 30 random pairs of new velocities were drawn and kept whenever each velocity lay in its robot's half-plane, 16,535 velocity pairs in all. For every one of them, the closest distance between the two robots within the time horizon was computed in closed form. The smallest value over all pairs is 1.005 m, above the contact distance of 1 m in every case, as the theory requires.

C. Symmetry and the keep-right rule

Without the keep-right rule, the benchmark fails. The left panel of Fig. 3 shows why: the robots head straight for the centre, slow down as their half-planes tighten, and stop in a ring around it, where they remain until the time limit. The reason is symmetry. For a robot on the circle, the other robots are arranged symmetrically about its line to the goal, so its half-planes are mirror images in pairs and its preferred velocity lies on the mirror axis. The linear program then has a symmetric feasible set and a symmetric objective, so its unique solution lies on the axis: the robot can slow down but never step aside. Every robot faces the same situation, so none of them moves aside, and the crowd reaches a symmetric standstill.

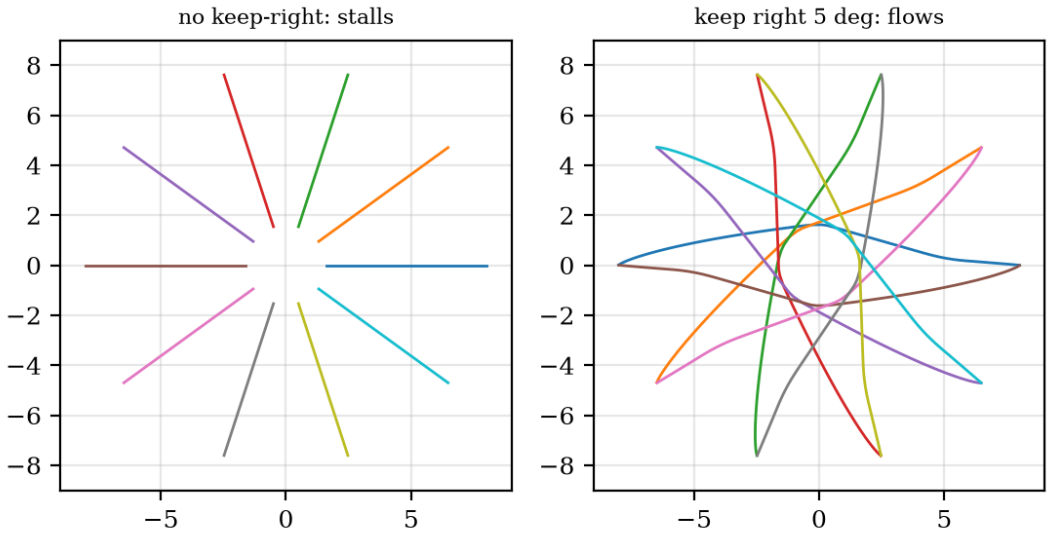


Fig. 3. Paths without the keep-right rule, where the robots stall in a ring around the centre (left), and with a keep-right angle of 5°, where they flow around it (right).

Turning every preferred velocity to the right breaks the mirror symmetry while keeping the rotational one: every robot now prefers to pass the centre on the same side, and the crowd organises itself into a roundabout. Table III shows how little bias is needed. One degree suffices to avoid the stall, although the crowd then takes 14.9 s to resolve; from 2° on, the swap completes within 2.6 s of the free-flight time. With 20° the robots keep a wider berth, 1.21 m at the closest, but take longer than with 10°, because the bias now distorts their paths noticeably even when no other robot is near.

TABLE III
EFFECT OF THE KEEP-RIGHT ANGLE (TEN ROBOTS)

Keep-right angle	Swap completed	Time	Closest approach
0°	no, stalls	–	1.000005 m
1°	yes	14.9 s	1.000003 m
2°	yes	13.3 s	1.0003 m
5° (demonstration)	yes	11.8 s	1.00003 m
10°	yes	11.1 s	1.0004 m
20°	yes	11.4 s	1.21 m

The bias has a price when a robot is alone. A robot whose velocity always makes a fixed angle θ_r with the direction to its goal follows a logarithmic spiral, whose length is that of the straight line divided by $\cos\theta_r$: longer by 0.4% for 5°, 1.5% for 10° and 6.4% for 20°. The demonstration uses 5° as a compromise between a negligible detour for a lone robot and a quick resolution of the crowd.

D. Crowd size

Table IV shows the benchmark with 4 to 24 robots. Up to 16 robots, the swap completes, in a time that grows from 10.8 s to 14.2 s as the roundabout becomes congested, and the closest approach is again essentially at contact. With 24 robots the crowd jams: the robots fill the space around the centre, and each one's best velocity within its half-planes is almost zero. Remarkably, the linear program never became infeasible, so the fallback of Section IV-D was never used: the jam is not a failure of the optimisation but a configuration in which every robot is doing the best that its half-planes allow. The time per step grows roughly with the square of the number of robots, from 0.19 ms for 4 robots to 6.4 ms for 24, because every robot considers every other one.

TABLE IV
EFFECT OF THE NUMBER OF ROBOTS (KEEP-RIGHT ANGLE 5°)

Robots	Swap completed	Time	Closest approach	Time per step
4	yes	10.8 s	1.003 m	0.19 ms
8	yes	11.3 s	1.001 m	0.73 ms
10	yes	11.8 s	1.00003 m	1.2 ms
16	yes	14.2 s	1.0000000004 m	2.8 ms
24	no, jams	–	1.000001 m	6.4 ms

VII. DISCUSSION

ORCA guarantees that robots do not collide; it does not guarantee that they reach their goals. Both failures observed here, the symmetric standstill and the jam of 24 robots, are configurations in which every robot's velocity is safe and locally optimal, yet the crowd makes no progress. The keep-right rule resolves the first by a convention that every robot follows, the same mechanism that resolves head-on encounters between pedestrians and vehicles; the second needs more than local reasoning, such as a planner that routes robots around congestion or a schedule that limits how many enter the central area at once.

The guarantee itself rests on assumptions that deserve to be stated. Every robot must run ORCA with the same time horizon and take its half of the avoidance; a robot that does not cooperate leaves the other with only half the needed correction. Each robot must know its neighbours' positions and velocities exactly and be able to change its own velocity instantaneously. Real robots have inertia and, often,

nonholonomic constraints, for which extensions of ORCA exist [6], [7]. The near-zero clearances of Tables II–IV show that, without a margin added to the robots' radii, small errors in any of these assumptions would turn grazing passes into collisions.

Finally, the cost per robot grows with the number of neighbours considered. The implementation considers all of them, which is quadratic in the crowd size overall; RVO2 restricts each robot to its nearest neighbours within a radius, found with a spatial index, which makes the method practical for crowds of thousands of simulated agents [5].

VIII. Conclusion

This report presented an implementation of optimal reciprocal collision avoidance and evaluated it on the circle-swap benchmark. Ten robots swap sides in 11.8 s, 10% above the time of a lone robot, without ever overlapping, and a check on 16,535 random velocity pairs confirmed the reciprocity guarantee on which the method's safety rests. The benchmark also exposed the method's blind spot, progress: perfect symmetry stalls the crowd, a keep-right bias of as little as one degree resolves it, and a crowd of 24 robots jams although every linear program stays feasible. The code, the tests and the script that regenerates every figure and number in this report are available at github.com/robofan45/orca-collision-avoidance.

Acknowledgment

The software and this report were developed with the assistance of an AI coding assistant (Claude, by Anthropic). All results were produced by the code in the repository and can be regenerated with scripts/make_figures.py.

References

[1] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *Int. J. Robot. Res.*, vol. 17, no. 7, pp. 760–772, 1998.

[2] J. van den Berg, M. Lin, and D. Manocha, "Reciprocal velocity obstacles for real-time multi-agent navigation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2008, pp. 1928–1935.

[3] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Robotics Research: The 14th International Symposium ISRR*, Springer Tracts in Advanced Robotics, vol. 70. Berlin, Germany: Springer, 2011, pp. 3–19.

[4] J. Snape, J. van den Berg, S. J. Guy, and D. Manocha, "The hybrid reciprocal velocity obstacle," *IEEE Trans. Robot.*, vol. 27, no. 4, pp. 696–706, 2011.

[5] J. van den Berg, S. J. Guy, J. Snape, M. C. Lin, and D. Manocha, "RVO2 Library: Reciprocal collision avoidance for real-time multi-agent simulation," 2011. [Online]. Available: https://gamma.cs.unc.edu/RVO2/

[6] J. Alonso-Mora, A. Breitenmoser, M. Rufli, P. Beardsley, and R. Siegwart, "Optimal reciprocal collision avoidance for multiple non-holonomic robots," in *Distributed Autonomous Robotic Systems*, Springer Tracts in Advanced Robotics, vol. 83. Berlin, Germany: Springer, 2013, pp. 203–216.

[7] D. Bareiss and J. van den Berg, "Generalized reciprocal collision avoidance," *Int. J. Robot. Res.*, vol. 34, no. 12, pp. 1501–1514, 2015.

[8] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *Int. J. Robot. Res.*, vol. 5, no. 1, pp. 90–98, 1986.

[9] D. Helbing and P. Molnár, "Social force model for pedestrian dynamics," *Phys. Rev. E*, vol. 51, no. 5, pp. 4282–4286, 1995.

[10] L. Wang, A. D. Ames, and M. Egerstedt, "Safety barrier certificates for collisions-free multirobot systems," *IEEE Trans. Robot.*, vol. 33, no. 3, pp. 661–674, 2017.

[11] Y. F. Chen, M. Liu, M. Everett, and J. P. How, "Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2017, pp. 285–292.

[12] R. Seidel, "Small-dimensional linear programming and convex hulls made easy," *Discrete Comput. Geom.*, vol. 6, no. 3, pp. 423–434, 1991.

[13] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars, *Computational Geometry: Algorithms and Applications*, 3rd ed. Berlin, Germany: Springer, 2008.

[14] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[15] A. Sakai, D. Ingram, J. Dinius, K. Chawla, A. Raffin, and A. Paques, "PythonRobotics: A Python code collection of robotics algorithms," arXiv:1808.10703, 2018.